Terraform

✅ What is Terraform?

Terraform is an open-source Infrastructure as Code (IaC) tool by HashiCorp that lets you define, provision, and manage cloud infrastructure using configuration files (usually in HCL – HashiCorp Configuration Language).

🔧 Why We Use Terraform?

- To automate infrastructure setup (servers, networks, databases).
- Make infrastructure version-controlled and reproducible.
- Multi-cloud support (AWS, Azure, GCP, etc.)
- Easy to scale, update, or destroy infrastructure.
- Declarative syntax (HCL)
- Modular and reusable code
- Tracks & changes (plan, apply & destroy)
- Strong community & ecosystem

⏳ Before vs After Terraform

Aspect	Before Terraform	After Terraform
Provisioning	Manual via UI/CLI/scripts	Automated via code
Consistency	Error-prone, hard to replicate	Consistent, version-controlled
Scalability	Time-consuming	Easy to scale or replicate
Change tracking	Difficult	Built-in plan & apply with diffs

🧠 Use Cases

- Cloud infrastructure provisioning (EC2, S3, VPC)
- Multi-environment deployment (dev, staging, prod)
- Immutable infrastructure setup
- CI/CD environment setup
- Disaster recovery infra setup

✅ Benefits

- **Declarative syntax** – define what, not how.
- **Reusability** – use modules.
- **Multi-cloud support**.
- **Safe changes** – plan before apply.
- **Team collaboration** – store in VCS like Git.

Akash Kumar

❌ Disadvantages

- Learning curve for HCL and state management.
- State file can be a **single point of failure** (needs remote backend).
- **Complex for dynamic resources** (limited logic in HCL).
- Terraform updates may break old code.

📦 Types of Terraform (Scopes)

1. **Terraform Open Source** – Free, CLI tool.
2. **Terraform Cloud** – Hosted service for remote runs, state, team features.
3. **Terraform Enterprise** – Self-hosted with advanced features (audit, RBAC).
4. **Providers** – AWS, Azure, GCP, Kubernetes, etc.
5. **Modules** – Reusable components.

✅ What is IaC (Infrastructure as Code)?

IaC means managing and provisioning **infrastructure using code**, instead of manual processes.

It ensures **automation, repeatability, and version control** for cloud infrastructure.

🧰 Types of IaC Tools

Type	Tools
Declarative	Terraform, CloudFormation, Pulumi
Imperative	Ansible, Chef, Puppet

1. **Declarative**: Describe what you want (e.g., a VM).
2. **Imperative**: Describe how to do it step-by-step.

📜 HCL Basics (Terraform Syntax)

1. Provision

- Use `terraform apply` to **create infrastructure**.

```
resource "aws_instance" "web" {
  ami = "ami-123456"
  instance_type = "t2.micro"
}
```

2. **Update**

- Change HCL code and run terraform apply → Terraform updates resources as needed.

3. Destroy

- Run terraform destroy to remove all managed infrastructure.

Akash Kumar

🔌 What is a Provider?

Plugin to interact with cloud/service (e.g., AWS, Azure, GCP).

```
provider "aws" {
  region = "us-east-1"
}
```

📥 Input Variables

Allow you to pass dynamic values into modules/configs.

```
variable "region" {
  default = "us-east-1"
}
```

📤 Output Variables

Used to show values after apply, or pass between modules.

```
output "instance_ip" {
  value = aws_instance.web.public_ip
}
```

🔧 Resource Attributes

Key-value pairs inside a resource block.

```
resource "aws_instance" "web" {
  ami           = "ami-123456"       # attribute
  instance_type = "t2.micro"         # attribute
}
```

🔗 1. Resource Dependencies

- Terraform automatically detects dependencies via references.

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "sub" {
  vpc_id = aws_vpc.main.id   # creates dependency
}
```

🗃️ Terraform State

1. Stores **current state of infrastructure** (in `.tfstate` file).
2. Needed to **track changes and manage resources**.

✅ Can be stored:

1. Locally (default)
2. Remotely (S3, Terraform Cloud, etc.)

📃 Terraform Commands

Command	Use
`terraform init`	Initialize Terraform project
`terraform plan`	Show changes before applying
`terraform apply`	Apply the planned changes
`terraform destroy`	Remove all infrastructure
`terraform validate`	Validate HCL syntax
`terraform fmt`	Format the code
`terraform show`	View current state
`terraform output`	View output values

🔄 Mutable vs Immutable Infrastructure

Type	Description
Mutable	Changes are made directly on existing infra
Immutable	Replaces old infra with new (clean deployment)

- **Terraform encourages immutable for safety and consistency.**

🔁 Terraform Lifecycle Meta-Argument

```
lifecycle {
  create_before_destroy = true
  prevent_destroy       = true
  ignore_changes        = [tags]
}
```

Akash Kumar

📚 6. Data Sources

- Read existing external resources (not create them).

```
data "aws_ami" "latest" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*"]
  }
}
```

🧩 7. Meta-Arguments

Used **inside resources** to control behavior:

- `provider`
- `count`
- `for_each`
- `depends_on`
- `lifecycle`

☁️ 8. AWS Basics (Terraform Terms)

AWS Concept	Terraform Resource
EC2	`aws_instance`
VPC	`aws_vpc`
Subnet	`aws_subnet`
S3 Bucket	`aws_s3_bucket`
Security Group	`aws_security_group`

🔐 9. IAM with Terraform

Create users, roles, policies:

```
resource "aws_iam_user" "example" {
  name = "demo-user"
}

resource "aws_iam_policy" "read_s3" {
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = ["s3:GetObject"]
      Effect = "Allow"
      Resource = "*"
    }]
  })
}
```

Akash Kumar

🔒 10. State Locking

- Prevents **multiple people from applying changes at the same time**.
- Usually managed with a remote **backend** (e.g., S3 + DynamoDB).

```
backend "s3" {
  bucket         = "my-terraform-state"
  key            = "prod/terraform.tfstate"
  region         = "us-east-1"
  dynamodb_table = "terraform-locks"
  encrypt        = true
}
```

⚙️ How Terraform Works with Shell Script

Shell scripts can be used to:

- Automate Terraform commands (`init`, `plan`, `apply`, `destroy`)
- Pass environment variables
- Control flow and logging
- Handle workspaces or modules

🧪 Example: Terraform with Shell Script

1. Terraform Configuration (main.tf)

```
provider "aws" {
  region = var.region
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = var.bucket_name
  acl    = "private"
}

variable "region" {}
variable "bucket_name" {}
```

2. Shell Script (deploy.sh)

```
#!/bin/bash

# Set variables
REGION="us-east-1"
BUCKET_NAME="my-demo-bucket-123456"

# Export environment variables
export TF_VAR_region=$REGION
export TF_VAR_bucket_name=$BUCKET_NAME

# Terraform commands
terraform init
terraform plan
terraform apply -auto-approve
```

Usage:

```
chmod +x deploy.sh
./deploy.sh
```

✅ This automates provisioning via Terraform using shell.

🛡️ Basic ACL Knowledge

ACL (Access Control List) defines who can access a resource and what they can do (read/write).

ACL in Terraform (e.g., for S3):

```
resource "aws_s3_bucket" "example" {
  bucket = "my-acl-bucket"
  acl    = "private"  # Options: private, public-read, public-read-write
}
```

ACL Options:

- private: Owner only
- public-read: Anyone can read
- authenticated-read: AWS users can read

📌 Key Points Summary

- Shell scripts automate Terraform workflows.
- Environment variables can inject values (TF_VAR_ prefix).
- ACL is used to define access rights to resources (e.g., S3).
- Ideal for CI/CD pipelines and infrastructure automation.

✅ What is ACL (Access Control List)?

ACL is a security mechanism that defines who can access a resource and what actions they can perform, like read, write, execute.

🔑 Key Concepts of ACL:

- **Subjects:** Who? (User, Group, Role)
- **Objects:** What? (File, Bucket, DB)
- **Permissions:** Allowed actions (read, write, delete)

📦 Types of ACL (Common Use)

ACL Type	Description
Discretionary (DACL)	Owner defines who gets what access
Mandatory (MACL)	System-enforced (e.g., by labels/policies)
Access vs Deny	Allow or explicitly block access

🧱 Example: AWS S3 Bucket ACL in Terraform

```
resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-secure-bucket"
  acl    = "private"  # other options below
}
```

Common S3 ACL Options:

- private: Only owner has full access
- public-read: Public can read
- public-read-write: Public can read/write (not recommended)
- authenticated-read: AWS-authenticated users can read

🔐 ACL vs IAM

ACL	IAM
Resource-level (like S3)	User/Role-level (fine-grained)
Simple access control	Policy-based, more scalable
Limited logic	JSON-based, conditional logic

Akash Kumar